UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



Redes de Computadoras
Trabajo Práctico Laboratorio N°5

Docente: Ing. Facundo Oliva Cuneo, Ing. Santiago Martín Henn
Año: 2026
Semestre: Segundo Semestre

Apellido y Nombre: *Bastía, Álvaro*
DNI: *44.783.012*
Apellido y Nombre: *Britez, Carlos*
DNI: *44.221.771*
Apellido y Nombre: *Cravero, Lorenzo*
DNI: *45.106.171*
Apellido y Nombre: *Genero, Sol*
DNI: *46.541.859*
Apellido y Nombre: *González, Tobías*
DNI: *46.080.084*
Apellido y Nombre: *Usandivaras, Hebe Azul*
DNI: *44.768.075*

Índice:

1)....3
a) 3
b) 3
c) 3
d) 4
a) 7
b) 8
c) 8
d) 9
e) 9
2)....10
a) 10
b) 10
c) 10
d) 10
e) 11
f) 11
a) 15
b) 15
c) 15
d) 15
3)....15
a) 15
b) 16
c) 16
Experimento TCP....16
Experimento UDP....16
a) 17
b) 17
c) 18
d) 18
e) 18
4)....18
a) 19

1)

Empezamos con la herramienta de red más conocida: ping. La vamos a usar como "generador de tráfico" para aprender a leer una captura. Investigar brevemente y documentar en el trabajo:

a)

¿Qué es ICMP y para qué se usa? ¿Transporta datos de aplicaciones como lo hacen TCP o UDP?

ICMP (Internet Control Message Protocol) es un conjunto de reglas de comunicación que los dispositivos utilizan para comunicar errores de transmisión de datos en una red. Se usa para ayudar a solucionar problemas permitiendo que los dispositivos se avisan entre sí cuando algo anda mal, como si no se puede acceder a un sitio web o si hay mucho tráfico que ralentiza las cosas

No transporta datos de aplicación como lo hacen TCP o UDP ya que estos son protocolos de transporte cuyo trabajo es mover datos de una aplicación a otra, en cambio, ICMP es un procedimiento de control que informa sobre el estado de la red, no para las aplicaciones.

b)

¿Qué relación tiene con IP? ¿Viaja dentro de IP, al lado de IP o debajo de IP? ¿Cómo sabe el receptor que el contenido de un paquete IP es ICMP?

La relación que tienen es que el ICMP es un protocolo integral y necesario de la suite IP (Protocolo de Internet) que está diseñado para gestionar errores y control operativo que el protocolo IP básico no proporciona por sí mismo. Básicamente los mensajes ICMP se encapsulan dentro de datagramas IP para poder viajar por la red, es decir, ICMP *necesita* la cabecera IP (direcciones de origen/destino, TTL, etc.) para que los routers sepan cómo enrutar sus mensajes.

Viaja *dentro de IP* (físicamente) pero está *al lado de IP* (arquitectónicamente) y no viaja debajo eso sería capa 2 o Ethernet.

El receptor lo sabe por el campo Protocol del encabezado IPv4, es un campo de 8 bits que indica qué protocolo *vive* en el payload o carga útil. Cuando su valor es 1, el stack sabe que debe entregar los datos al módulo ICMP.

c)

¿Qué hace ping? ¿Qué son un Echo Request y un Echo Reply? ¿Qué campos de ICMP permiten distinguirlos?

El comando ***ping*** es una utilidad disponible en la mayoría de los sistemas operativos (Windows, Linux, macOS) usada para diagnósticos de red. Si se escribe

ping seguido de un nombre de host o dirección IP (por ejemplo, ping example.com), el comando inicia una serie de mensajes ICMP Echo Request al objetivo especificado.

El ***Echo Request*** es un mensaje de control que se envía a un host con la expectativa de recibir de él un Echo Reply. Esto es conocido como Ping y es una utilidad del protocolo ICMP, subprotocolo de IP. Todo host debe responder a un *Echo Request* con un *Echo Reply* que contenga exactamente los mismos datos que el primero por lo tanto el ***Echo Replay*** es un mensaje generado como respuesta a un mensaje *Echo Request.*

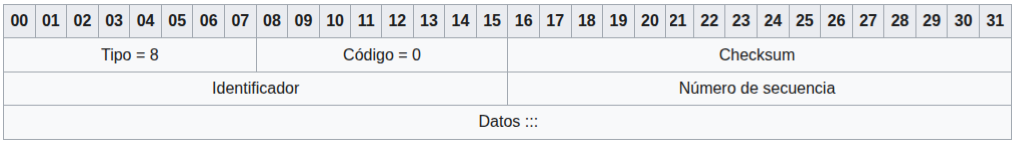
Los campos permiten distinguirlos son Identifier y Sequence Number estos permiten al emisor correlacionar cada solicitud con su respuesta específica y asegurarse que los paquetes coincidan correctamente.

d)

¿Qué información mínima contiene un mensaje ICMP de tipo Echo?

Debe contener como mínimo 8 bytes de cabecera ICMP y estos campos: Tipo, Código, Checksum, Identificador, Número de Secuencia.

Ejemplo, Formato del mensaje:



- En tipo es 8 si es Echo Request o 0 en Echo Reply.
- El código siempre es 0.
- La Checksum es la suma de comprobación de todo el mensaje ICMP.
- Identificador para emparejar solicitud/respuesta (puede ser 0).
- El número de secuencia es para emparejar (puede ser 0).

Averiguar la configuración de red de alguna de las computadoras del grupo: dirección IPv4, máscara, gateway por defecto y dirección MAC de la interfaz que usan (Wi-Fi o cableada).

Interfaz Wi-Fi utilizada: wlp2s0
Dirección MAC: 48:d2:24:3e:c1:75
Dirreccion IPv4: 192.168.1.3
Mascara: 255.255.255.0
Gateway por defecto: 192.168.1.1

Captura de pantalla de la terminal:

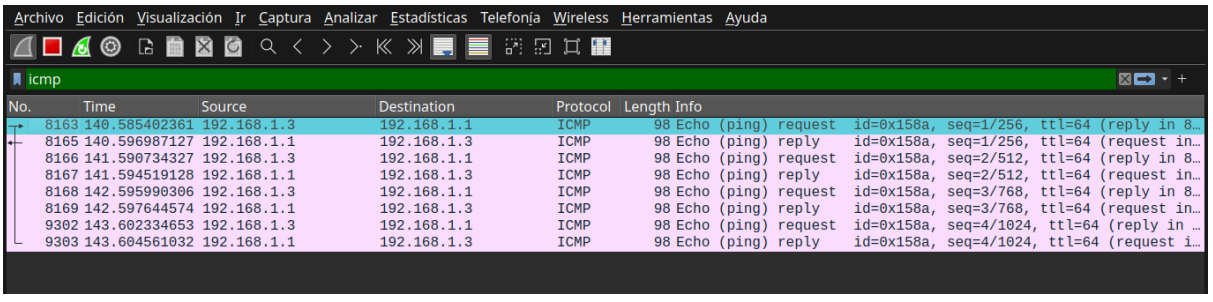
```
solgenero@pop-os:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp3s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 08:9e:01:4f:c5:69 brd ff:ff:ff:ff:ff:ff
3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 48:d2:24:3e:c1:75 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.3/24 brd 192.168.1.255 scope global dynamic noprefixroute wlp2s0
       valid_lft 80272sec preferred_lft 80272sec
    inet6 fe80::5abc:d587:6f64:726/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
solgenero@pop-os:~$ ip route
default via 192.168.1.1 dev wlp2s0 proto dhcp src 192.168.1.3 metric 600
192.168.1.0/24 dev wlp2s0 proto kernel scope link src 192.168.1.3 metric 600
```

Abrir Wireshark, elegir la interfaz por la que salen a Internet (Wi-Fi o Ethernet; la que muestra actividad en el gráfico) y comenzar la captura. En una terminal, hacer ping a la IP del gateway por defecto y luego a una IP de Internet:

Ping a la IP del Gateway por defecto:

```
solgenero@pop-os:~$ ping -4 -c 4 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=11.6 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=3.83 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=1.70 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=2.25 ms

--- 192.168.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3017ms
rtt min/avg/max/mdev = 1.697/4.844/11.606/3.980 ms
solgenero@pop-os:~$
```



Ping a una IP de internet:

```
solgenero@pop-os:~$ ping -4 -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=13.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=13.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=14.4 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=19.5 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3015ms
rtt min/avg/max/mdev = 13.687/15.359/19.527/2.420 ms
solgenero@pop-os:~$
```

36224	341.942931231	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=1/256, ttl=64 (reply in 3…
36225	341.956686968	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=1/256, ttl=118 (request i…
36234	342.947934549	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=2/512, ttl=64 (reply in 3…
36235	342.961594591	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=2/512, ttl=118 (request i…
36236	343.953068032	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=3/768, ttl=64 (reply in 3…
36237	343.967435042	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=3/768, ttl=118 (request i…
36238	344.957973474	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=4/1024, ttl=64 (reply in …
36239	344.977456338	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=4/1024, ttl=118 (request …

Filtros utilizados en el wireshark:

icmp.type == 8 (solo Echo Request)

icmp.type == 8

No.	Time	Source	Destination	Protocol	Length	Info
8163	140.585402361	192.168.1.3	192.168.1.1	ICMP	98	Echo (ping) request id=0x158a, seq=1/256, ttl=64 (reply in 8…
8166	141.590734327	192.168.1.3	192.168.1.1	ICMP	98	Echo (ping) request id=0x158a, seq=2/512, ttl=64 (reply in 8…
8168	142.595990306	192.168.1.3	192.168.1.1	ICMP	98	Echo (ping) request id=0x158a, seq=3/768, ttl=64 (reply in 8…
9302	143.602334653	192.168.1.3	192.168.1.1	ICMP	98	Echo (ping) request id=0x158a, seq=4/1024, ttl=64 (reply in …
36224	341.942931231	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=1/256, ttl=64 (reply in 3…
36234	342.947934549	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=2/512, ttl=64 (reply in 3…
36236	343.953068032	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=3/768, ttl=64 (reply in 3…
36238	344.957973474	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=4/1024, ttl=64 (reply in …

icmp.type == 0 (solo Echo Reply)

icmp.type == 0

No.	Time	Source	Destination	Protocol	Length	Info
8165	140.596987127	192.168.1.1	192.168.1.3	ICMP	98	Echo (ping) reply id=0x158a, seq=1/256, ttl=64 (request in…
8167	141.594519128	192.168.1.1	192.168.1.3	ICMP	98	Echo (ping) reply id=0x158a, seq=2/512, ttl=64 (request in…
8169	142.597644574	192.168.1.1	192.168.1.3	ICMP	98	Echo (ping) reply id=0x158a, seq=3/768, ttl=64 (request in…
9303	143.604561032	192.168.1.1	192.168.1.3	ICMP	98	Echo (ping) reply id=0x158a, seq=4/1024, ttl=64 (request i…
36225	341.956686968	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=1/256, ttl=118 (request i…
36235	342.961594591	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=2/512, ttl=118 (request i…
36237	343.967435042	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=3/768, ttl=118 (request i…
36239	344.977456338	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=4/1024, ttl=118 (request …

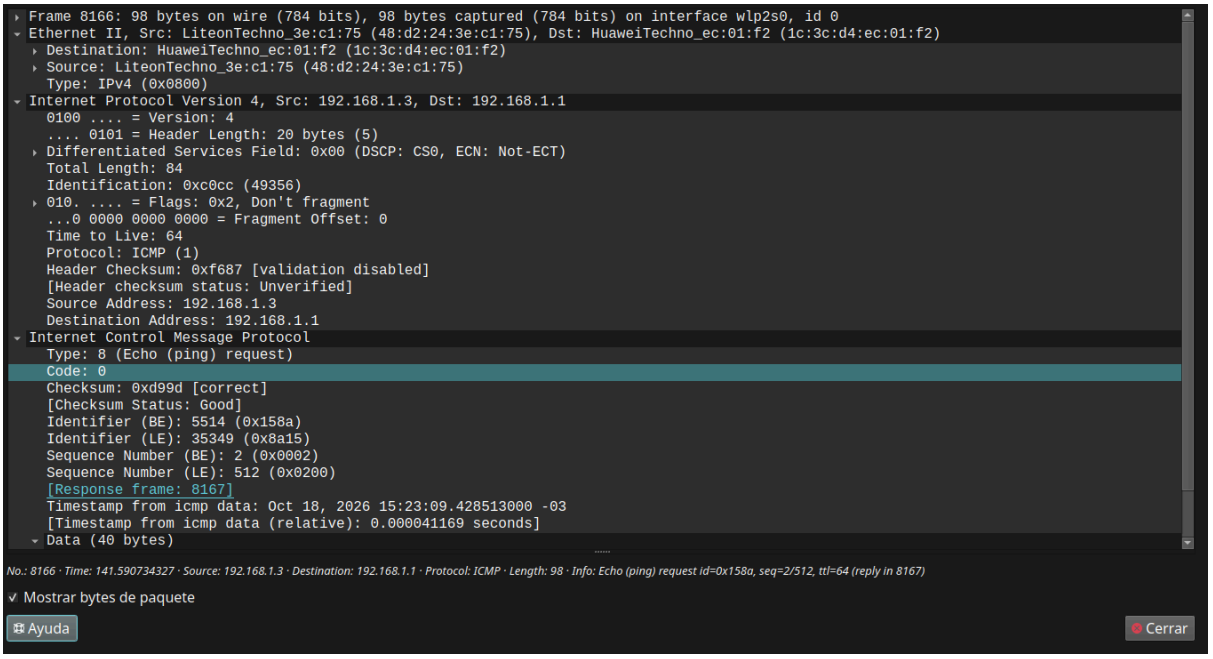
icmp && ip.addr == 8.8.8.8 (el intercambio con esa IP (en ambos sentidos))

icmp && ip.addr == 8.8.8.8

No.	Time	Source	Destination	Protocol	Length	Info
36224	341.942931231	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=1/256, ttl=64 (reply in 3…
36225	341.956686968	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=1/256, ttl=118 (request i…
36234	342.947934549	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=2/512, ttl=64 (reply in 3…
36235	342.961594591	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=2/512, ttl=118 (request i…
36236	343.953068032	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=3/768, ttl=64 (reply in 3…
36237	343.967435042	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=3/768, ttl=118 (request i…
36238	344.957973474	192.168.1.3	8.8.8.8	ICMP	98	Echo (ping) request id=0x16b0, seq=4/1024, ttl=64 (reply in …
36239	344.977456338	8.8.8.8	192.168.1.3	ICMP	98	Echo (ping) reply id=0x16b0, seq=4/1024, ttl=118 (request …

Seleccionar un Echo Request y desplegar el panel de detalles (el del medio). Identificar las capas que muestra Wireshark y completar la siguiente tabla:

Capa (como la nombra Wireshark)	Dirección/identificador origen	Dirección/identificador destino	¿Qué campo indica qué protocolo viene "adentro"?
Ethernet II	48:d2:24:3e:c1:75	1c:3c:d4:ec:01:f2	IPv4 (0x0800)
Internet Protocol Version 4	192.168.1.3	192.168.1.1	ICMP (1)
Internet Control Message Protocol	Identifier: 5514 (0x158a)	Sequence Number: 2	Type: 8 (Echo Request)
Datos / payload	-	-	-



Responder, justificando con la captura:

a)

La MAC destino del Echo Request enviado a 8.8.8.8, ¿es la MAC de 8.8.8.8? ¿De qué equipo es? Compárenla con la MAC destino del ping al gateway. ¿Qué conclusión sacan sobre el alcance de una dirección MAC frente al de una dirección IP?

No es la MAC de 8.8.8.8 .Es la dirección MAC del Gateway por defecto 1c:3c:d4:ec:01:f2 .La conclusión que sacamos es que las direcciones MAC tienen un alcance local (Capa 2) y las direcciones IP tiene un alcance global (Capa 3).

b)

Comparen un Echo Request con su Echo Reply (Wireshark los vincula en el campo [Response frame: …]). Hagan una lista de los campos que cambian y de los que se mantienen en Ethernet, IP e ICMP. ¿Por qué tiene sentido cada cambio? ¿Por qué el identificador y el número de secuencia se mantienen?

- Ethernet II:
  - + En *Echo Request*:
    Src: 48:d2:24:3e:c1:75 , Dst: 1c:3c:d4:ec:01:f2
  - + En *Echo Replay*:
    Src: 1c:3c:d4:ec:01:f2, Dst: 48:d2:24:3e:c1:75

  Se invierten el origen y el destino ya que la respuesta debe ir de la MAC del router a la MAC de la computadora.

- IP:
  - + En *Echo Request:*
    Src Address: 192.168.1.3, Dst Address: 192.168.1.1
  - + En *Echo Replay:*
    Src Address: 192.168.1.1, Dst Address: 192.168.1.3

  Se invierten el origen y el destino ya que la respuesta debe ir de la MAC del router a la MAC de la computadora.

- ICMP:
  - + En *Echo Request:* Type 8
  - + En *Echo Replay:* Type 0

  Cambian ya que el sistema tiene que saber que en Echo Replay no es una solicitud sino una confirmación de la respuesta a la petición del Echo Request.

El *identificador* y el *número de secuencia* no cambian ya que se necesitan para poder vincular y emparejar de manera más exacta la respuesta recibida con la solicitud de envío.

c)

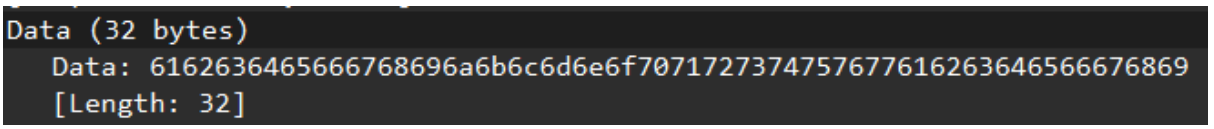
¿Dónde está el payload de ping? ¿Cuántos bytes tiene y qué contiene? ¿Es igual en el Reply? Si en el grupo hay una computadora con Windows y otra con Linux, compárenlos: ¿qué les sugiere que sean distintos?

El payload se encuentra en la parte de *Data,* tiene 40 bytes (en Linux) que contienen una secuencia de caracteres ascii que verifican que el paquete llegó completo. En Reply es igual.

Linux:



Windows:



Nos sugiere que sean distintos primero porque ambos sistemas operativos tiene una cantidad de bytes distinta, esto es así porque el protocolo ICMP no exige un tamaño específico del payload distinta, por otro lado cada distribución implementa la herramienta *ping* eligiendo un patrón y tamaño de datos que prefiera ya que lo que se necesita es una secuencia que pueda verificar que no se corrompieron datos en la trayectoria

d)

¿Qué valor de TTL tiene el Echo Request que ustedes enviaron? ¿Y el Reply que llegó de 8.8.8.8? ¿Por qué no son iguales?

TTL del Echo Request que enviamos: 64
TTL del Echo Replay que llego de 8.8.8.8 : 118

¿Qué hace un router con el TTL?

El TTL(Time To Live) hace referencia al número máximo de saltos o routers que un paquete de datos puede atravesar antes de ser descartado. Asegura que los datos no se queden atrapados en bucles de red y que la transmisión sea más eficiente y segura.

Funcionamiento:

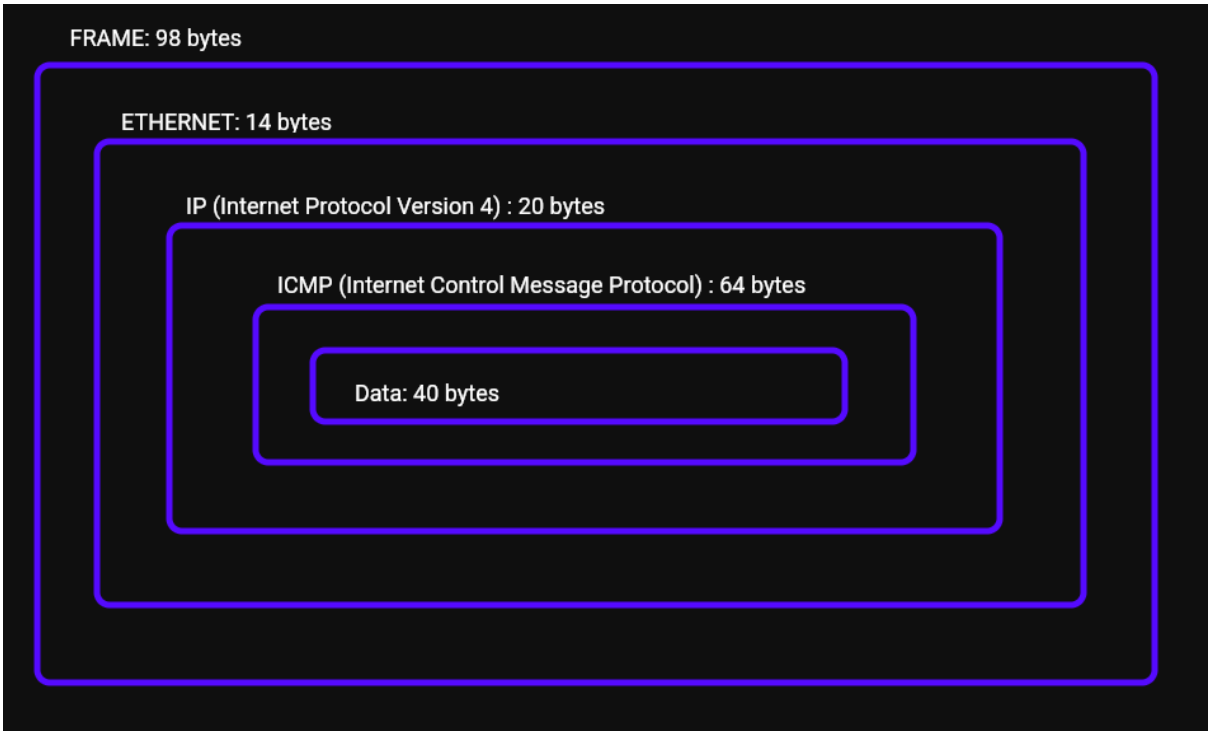
- El dispositivo de origen asigna un valor inicial de TTL (por ejemplo, 64, 128 ó 255, según el sistema operativo).
- Cada router que recibe el paquete resta 1 al valor TTL.
- Si el TTL llega a cero, el paquete se descarta y el router envía un mensaje ICMP de error al emisor.

De esta forma, se evita que los paquetes queden circulando sin destino en la red.

Podemos decir entonces que no son iguales ya que cada sistema operativo arranca con un paquete TTL distinto dependiendo la distribución (Linux, WIndows, etc) y a medida que va pasando por los routers intermedios de internet va reduciendo su valor en 1 hasta llegar a la terminal de la computadora

e)

Dibujen la encapsulación del paquete que eligieron como "cajas dentro de cajas", indicando para cada caja qué tamaño en bytes tiene según Wireshark.



2)

En el punto anterior la trama Ethernet ya tenía una MAC destino. Pero Ping solo conoce una IP. ¿De dónde salió esa MAC? Investigar brevemente:

a)

¿Qué problema resuelve ARP? ¿En qué capa lo ubicarían y por qué es discutible?

ARP es un protocolo fundamental que traduce direcciones IP lógicas en direcciones MAC físicas. Su función principal es asociar una dirección IP dinámica con la dirección de hardware fija de una máquina para garantizar que los paquetes de datos lleguen al dispositivo correcto.

El problema que resuelve es la discontinuidad entre dos esquemas de direccionamiento:

- La capa de Red (L3) opera con direcciones IP lógicas y jerárquicas pensada para el enrutado.
- La capa de Enlace (L2) entrega tramas usando direcciones MAC físicas de 48 bits que solo tienen sentido dentro de un segmento local.

Descubre dinámicamente la dirección MAC correspondiente a una IP dada, permitiendo que el tramo se dirija al hardware correcto (El punto mencionado en el enunciado del ejercicio).

b)

¿Qué es un ARP Request y un ARP Reply? ¿A quién se envía cada uno?

ARP Request es un mensaje Broadcast enviado a todos los dispositivos de la red local. **ARP Reply** es un mensaje Unicast enviado **solo al dispositivo que hizo la solicitud**.

c)

¿Qué es la caché ARP y por qué existe?

Es una tabla temporal en memoria de cada dispositivo que almacena los pares **IP - MAC** resueltos. Cada entrada tiene un **TTL** (time-to-live) y se elimina automáticamente al expirar para evitar asociaciones obsoletas.

d)

Traten de responder con sus palabras: "Tengo la IP de una máquina de mi red local. ¿Cómo sé a qué dirección MAC debo enviarle la trama?"

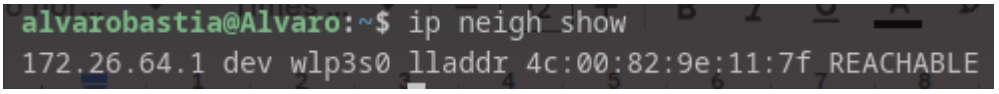
En teoría no sabes, de eso se encarga el protocolo ARP. El switch tiene la tabla de direccionamiento que se encarga de direccionar el MAC.

e)

Ver la caché ARP de su computadora y buscar la entrada del gateway:

```
# Windows
arp -a

# Linux
ip neigh show
```



¿La MAC asociada al gateway coincide con la MAC destino que vieron anteriormente?

f)

Generar tráfico ARP y capturarlo. Capturen en la interfaz Wi-Fi/Ethernet con el filtro arp. Como las entradas quedan guardadas en la caché, un ping a un equipo con el que ya hablaron puede no generar ningún ARP. Tienen varias formas de provocarlo (elijan la que puedan usar):

Opción A (no requiere permisos de administrador): hacer ping a una IP de su misma subred que no esté en uso. Por ejemplo, si su IP es 192.168.1.37/24, prueben 192.168.1.253 (antes verifiquen con arp -a / ip neigh que no aparezca).

```
# Windows
ping -4 -n 2 192.168.1.253

# Linux
ping -4 -c 2 192.168.1.253
```

Opción B (requiere administrador; el cambio es temporal): borrar solo la entrada del gateway y volver a hacerle ping. La entrada se vuelve a aprender sola en cuanto se necesita; no rompe la configuración de red.

```
# Windows (cmd o PowerShell "Ejecutar como administrador")
arp -d <IP_del_gateway>
ping -4 -n 2 <IP_del_gateway>

# Linux (reemplazar wlan0 por el nombre de su interfaz, ver "ip addr")
sudo ip neigh del <IP_del_gateway> dev wlan0
ping -4 -c 2 <IP_del_gateway>
```

Opción C: hacer ping a un equipo de la LAN con el que no hayan hablado recientemente (por ejemplo, la computadora del compañero/a).

Analizar un ARP Request y su ARP Reply. Para cada uno completar una tabla como la siguiente:

Si realizamos la prueba con la opción A, al hacer ping a una subred que no esté en uso, la IP utilizada es inexistente, por lo que solo tendría varios Requests sin Reply:

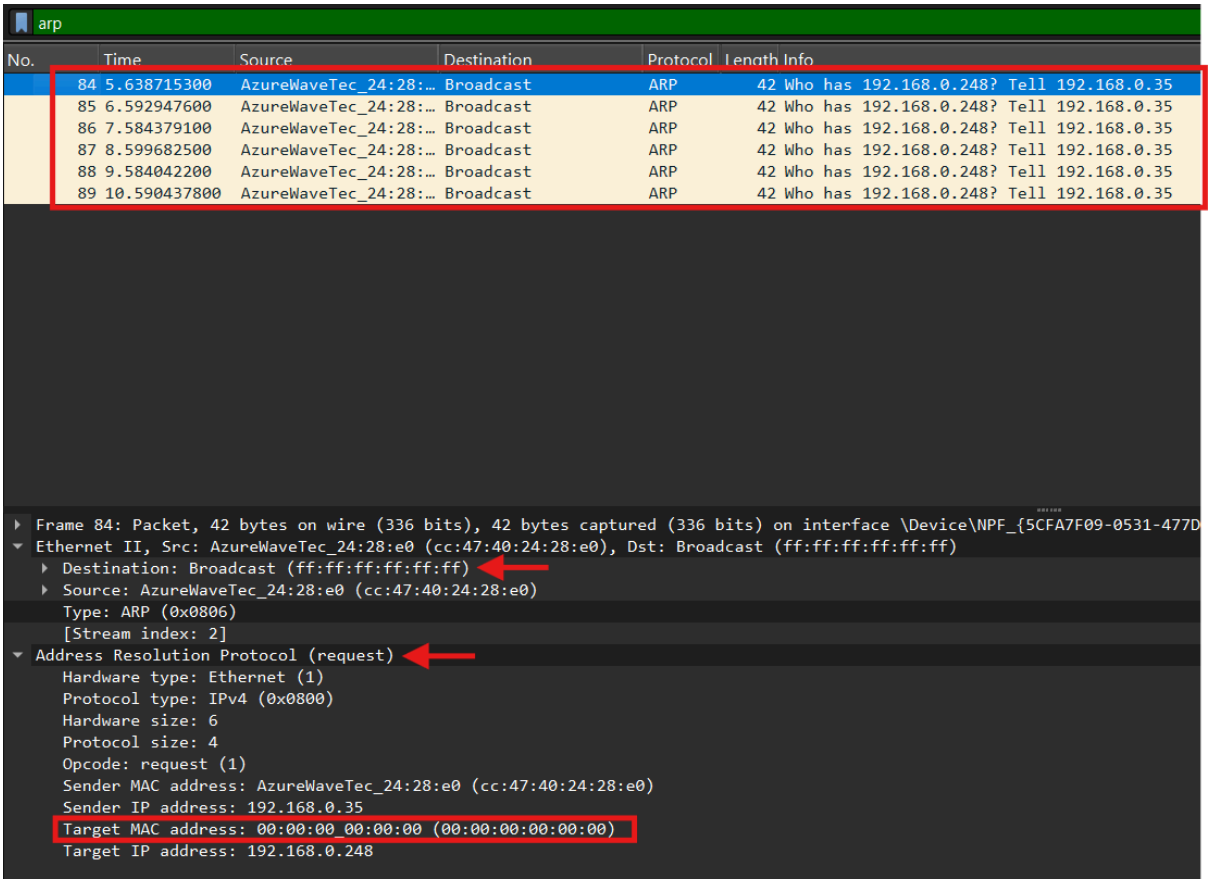
Para esta opción, utilizamos la IP “192.168.0.248” y realizamos el procedimiento correspondiente, obteniendo como resultado:

➔ PowerShell:

```
PS C:\Users\Lorenzo>    ping -4 -n 2 192.168.0.248

Haciendo ping a 192.168.0.248 con 32 bytes de datos:
Respuesta desde 192.168.0.35: Host de destino inaccesible.
Respuesta desde 192.168.0.35: Host de destino inaccesible.
```

➔ Wireshark:



Por lo tanto, para poder completar la tabla, nos basamos en la opción B, donde en este caso si tendremos Request y Reply: el Request va directo a Broadcast y el Reply va directo a la MAC del emisor que realizó el request:

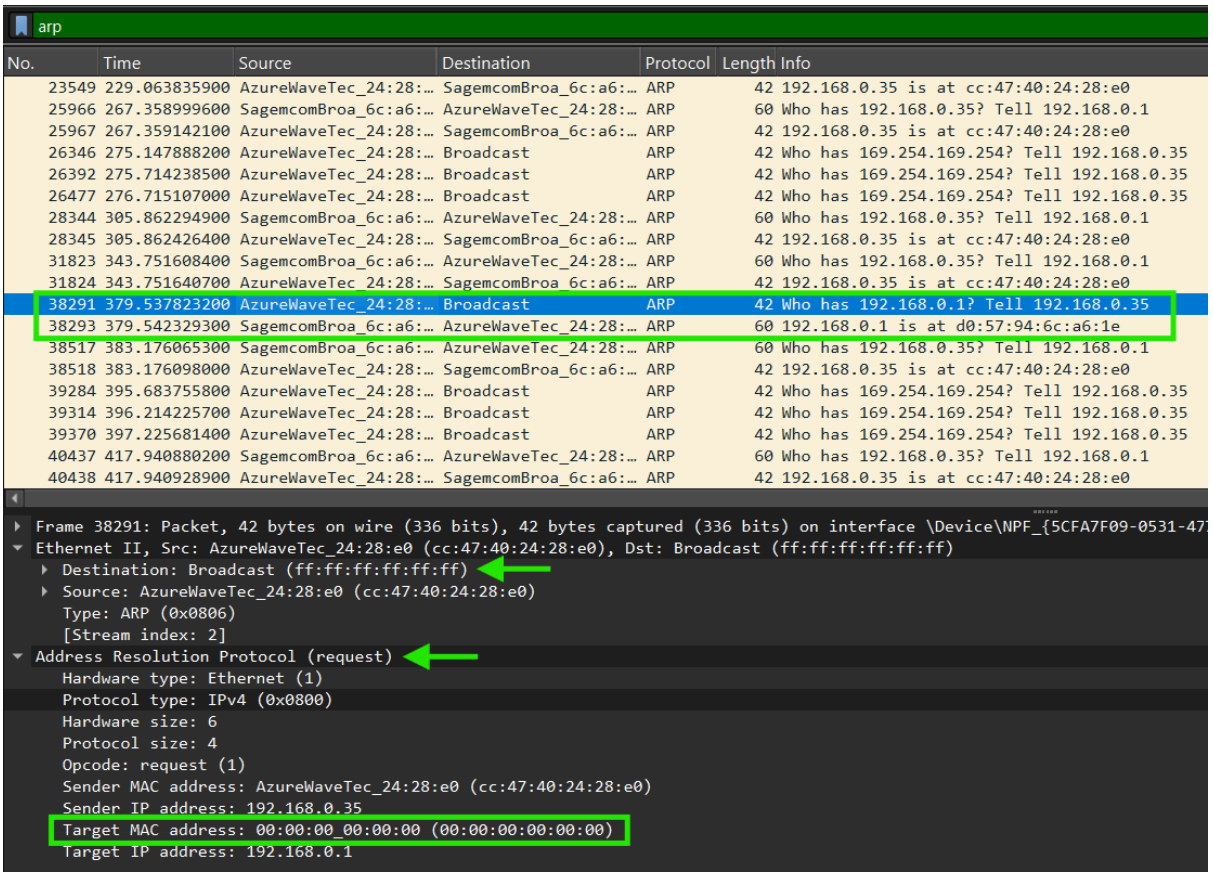
➔ PowerShell:

```
PS C:\WINDOWS\system32> ping -4 -n 2 192.168.0.1

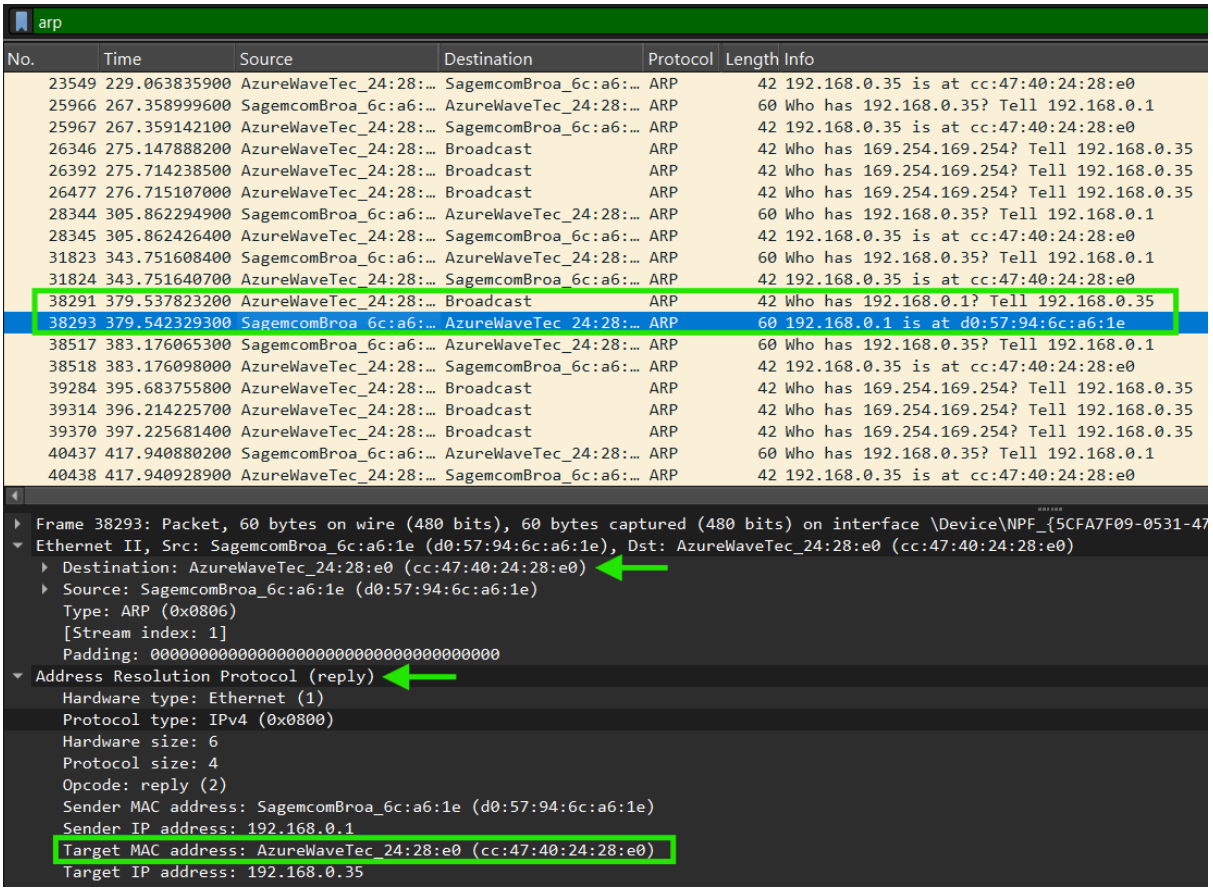
Haciendo ping a 192.168.0.1 con 32 bytes de datos:
Respuesta desde 192.168.0.1: bytes=32 tiempo=4ms TTL=64
Respuesta desde 192.168.0.1: bytes=32 tiempo=5ms TTL=64

Estadísticas de ping para 192.168.0.1:
    Paquetes: enviados = 2, recibidos = 2, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 4ms, Máximo = 5ms, Media = 4ms
```

➔ Wireshark: (Request)



➔ Wireshark: (Reply)



Campo	ARP Request	ARP Reply
MAC destino (encabezado Ethernet)	ff:ff:ff:ff:ff:ff	cc:47:40:24:28:e0
MAC origen (encabezado Ethernet)	cc:47:40:24:28:e0	d0:57:94:6c:a6:1e
Opcode	1	2
Sender MAC address	cc:47:40:24:28:e0	d0:57:94:6c:a6:1e
Sender IP address	192.168.0.35	192.168.0.1
Target MAC address	00:00:00:00:00:00	cc:47:40:24:28:e0
Target IP address	192.168.0.1	192.168.0.35

Y responder:

a)

¿Por qué el Request va a una dirección broadcast y el Reply no? ¿Qué valor tiene Target MAC address en el Request y por qué?

Esto es así porque, en Request, la PC **conoce la IP de destino** (porque la definimos nosotros) **pero no la MAC** a la que pertenece, por lo que sin esta, la única opción que le queda es mandar la trama de la IP a todos los equipos de la LAN (Broadcast) para que todos la procesen y solo el que tenga la IP correspondiente la responda. En el Reply esto no es así porque **el receptor del request ya tiene la IP y la MAC del emisor**, por lo que responder mediante broadcast sería innecesario, ya que puede responder al emisor en unicast directamente a esa MAC, sin molestar a los demás equipos.

El valor de la Target MAC address en el Request es 00:00:00:00:00:00, debido a que, como se mencionó anteriormente, no se conoce la MAC del receptor con la IP que definimos, por lo que este valor **se autocompleta con valores en 0** . En el Reply, Target MAC address si conoce la MAC del emisor, por lo que el valor de ésta será el que irá en este caso.

b)

En el encabezado Ethernet de la trama ARP, ¿qué valor tiene el campo Type? ¿Hay un encabezado IP? ¿Qué les dice eso sobre dónde "vive" ARP?

El valor del campo Type es 0x0806 y no hay encabezado IP, ya que al hacer las pruebas, las únicas secciones que se aprecian en Wireshark son las de “Ethernet II” y “Address Resolution Protocol”, **sin “Internet Protocol”**, sección que indica que la trama tiene encabezado IP, por lo que la ARP viaja directamente sobre Ethernet sin pasar por IP, lo cual tiene sentido en base a lo visto en las pruebas, ya que nos permite poder armas tramas cuando todavía no se puede enviar un paquete IP.

Esto nos da a entender que la ARP vive entre la capa de enlace (capa 2) y la capa de red (capa 3), ya que puede servir de unión para traducir direcciones IP a MAC.

c)

Con la Opción A (IP inexistente): ¿cuántos ARP Request aparecieron? ¿Hubo Reply? ¿Apareció algún ICMP Echo Request en la captura? Expliquen por qué.

Realizando las pruebas con la opción A, aparecieron 6 ARP Request, debido a que como la PC no recibe respuesta del request enviado, reintenta varias veces. En este caso en particular, como se explicó anteriormente, no hay Reply, debido a que, como el reply solo lo envía el receptor al que le pertenece la IP consultada, y en este caso la IP a la que se envía el request no está siendo utilizada, no hay nadie que conteste ese envío.

Además, no se tuvo ningún ICMP Echo Request en la captura, debido a que para que éste se pueda dar, la PC necesita poder armar una trama Ethernet con la MAC de destino, cosa imposible al ser ésta inexistente, por que la ARP falla localmente antes de que el paquete ICMP llegue a salir de la interfaz (por eso se puede ver en el PowerShell que dice “Host de destino inaccesible”).

d)

Volvieron a hacer ping al mismo destino un minuto después: ¿apareció ARP de nuevo? Revisen la caché ARP. ¿Qué ventaja tiene la caché y qué problema podría causar si una entrada quedara vieja?

Al volver a hacer las pruebas pasado un minuto aprox, apareció el ARP nuevamente. Investigando, esto se debe a que las entradas de la caché ARP son temporales (de aprox unas decenas de segundos si no se usan), y pasado este tiempo hay que revalidarlas nuevamente.

La ventaja que ofrece la caché es que nos evita tener que hacer una ARP Request por cada paquete que se quiere enviar, lo que reduce el tráfico de la red, baja la latencia y evita que todos los equipos LAN deban procesar todas las consultas si no son para ellos (cosa que ocurre en la opción A del ejercicio).

El problema que presenta ésta ante una entrada vieja es que, si el dispositivo al que se le está haciendo el request cambia de MAC pero mantiene la misma IP, la caché seguirá enviando tramas a la MAC original, lo que podría ocasionar pérdida de paquetes o que estos vayan al equipo equivocado.

3)

Hasta ahora observamos protocolos que usa el sistema operativo. Ahora vamos a comunicar dos procesos (dos programas) sin escribir código todavía. ncat es un programa que toma lo que escribimos en el teclado y lo manda por la red (y viceversa). La idea es empezar a generar tráfico cada vez más a bajo nivel (primero generamos tráfico con Packet Tracer, luego con la terminal del SO, ahora con un software a través de la CLI y finalmente ejecutaremos algunos scripts). Descargar netcat! para linux es bastante directo:

```
sudo apt install netcat-openbsd
```

Investigar brevemente:

a)

¿Qué significa "establecer una conexión"? ¿Dónde "existe" una conexión TCP: en los cables, en los routers o en los extremos?

En TCP, “establecer una conexión” significa que dos procesos se sincronizan y preparan el estado necesario para intercambiar datos. Esto se logra por medio del three-way handshake:

1. **SYN:** el cliente envía un paquete con la bandera SYN al servidor, indicando que desea establecer una conexión y le envía su número de secuencia inicial aleatorio (ISN).
2. **SYN-ACK:** el servidor recibe el paquete y responde con un mensaje SYN-ACK.
3. **ACK:** el cliente recibe el paquete SYN-ACK del servidor y envía otro de vuelta para confirmar, quedando establecida la conexión.

La conexión TCP “existe” en los extremos ya que son los sistemas operativos de cada host los que mantienen el estado de la misma en su RAM (números de secuencia, tamaño de la ventana de datos, los puertos). Los cables sólo transportan señales y los routers sólo reenvían paquetes mirando la IP de destino.

b)

¿Qué es un puerto? ¿Qué identifica el par (IP, puerto)?

Mientras que la dirección IP identifica al equipo en la red, un puerto es un número de 16 bits que utiliza la capa de transporte para saber a qué proceso dentro de un host le corresponde cada segmento o datagrama que llega.

El par (IP, puerto) identifica de forma única un punto de conexión para una aplicación específica en la red, permitiendo que los datos viajen no solo hasta la máquina correcta, sino directamente hasta el proceso que los está esperando.

c)

¿Qué significa que un proceso esté "escuchando" en un puerto?

Que un proceso esté “escuchando” en un puerto, significa que el programa le pidió al sistema operativo (con blind y listen) que le reserve ese puerto y le avise si llegan intentos de conexión, encontrándose activo y en estado de espera, listo para aceptar conexiones entrantes de otros dispositivos de la red. Si llega un SYN para ese puerto, el SO completa el handshake y le entrega la conexión al proceso (con accept), y si no, el SO responde con un RST y el cliente recibe un “connection refused”.

Importante: capturar en loopback. En esta parte cliente y servidor corren en la misma computadora y se comunican por 127.0.0.1 o localhost. Ese tráfico no sale por la placa Wi-Fi: en Wireshark hay que elegir la interfaz de loopback:

Windows (Npcap): Adapter for loopback traffic capture.
Linux: lo.

Van a notar que, según el sistema operativo, la captura en loopback no muestra un encabezado Ethernet o lo muestra con MAC 00:00:00:00:00:00.

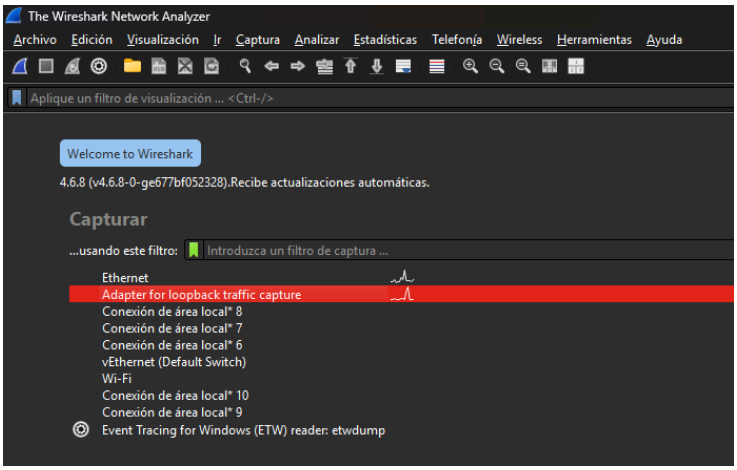
Experimento TCP.

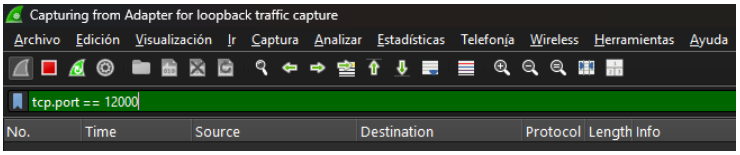
Iniciar la captura en loopback con el filtro tcp.port == 12000. Abrir dos terminales:

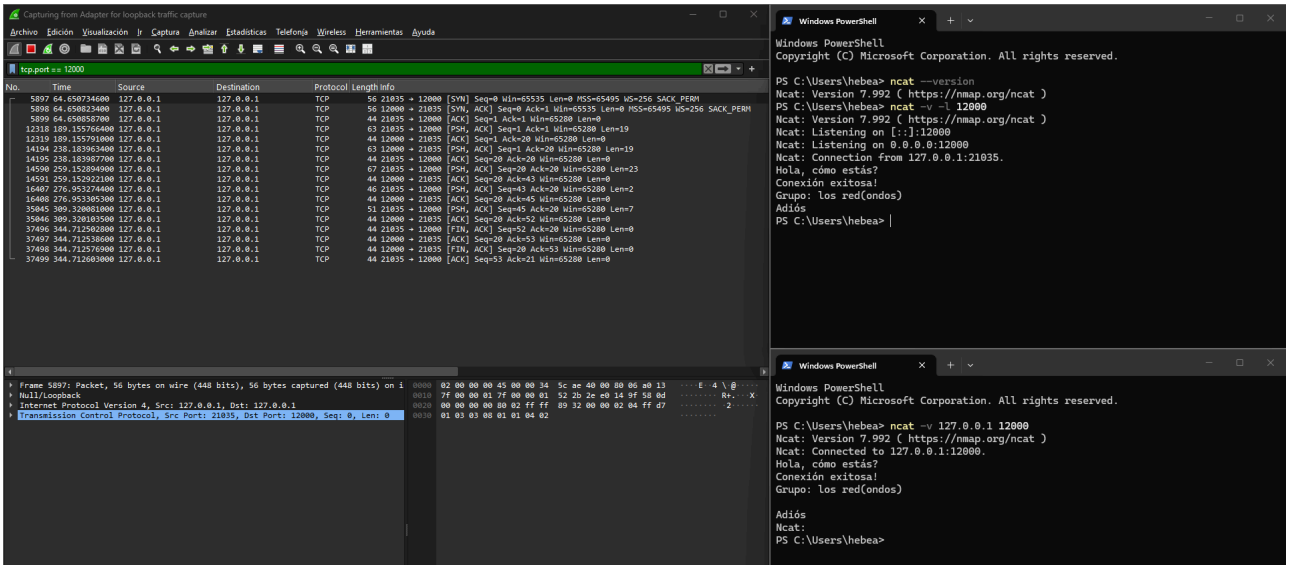
```
# Terminal A (servidor): escucha en el puerto TCP 12000
ncat -v -l 12000
```

```
# Terminal B (cliente): se conecta al servidor
ncat -v 127.0.0.1 12000
```

Escriban una línea en la terminal B y presionen Enter; luego respondan desde la A. Manden 2 o 3 mensajes en cada sentido. Finalmente cierren el cliente (Ctrl+C en B) y detengan la captura. Capturen el paquete con Wireshark, recuerden filtrar con tcp.port == 12000 así no se les hace una ensalada de paquetes en el sniffer.







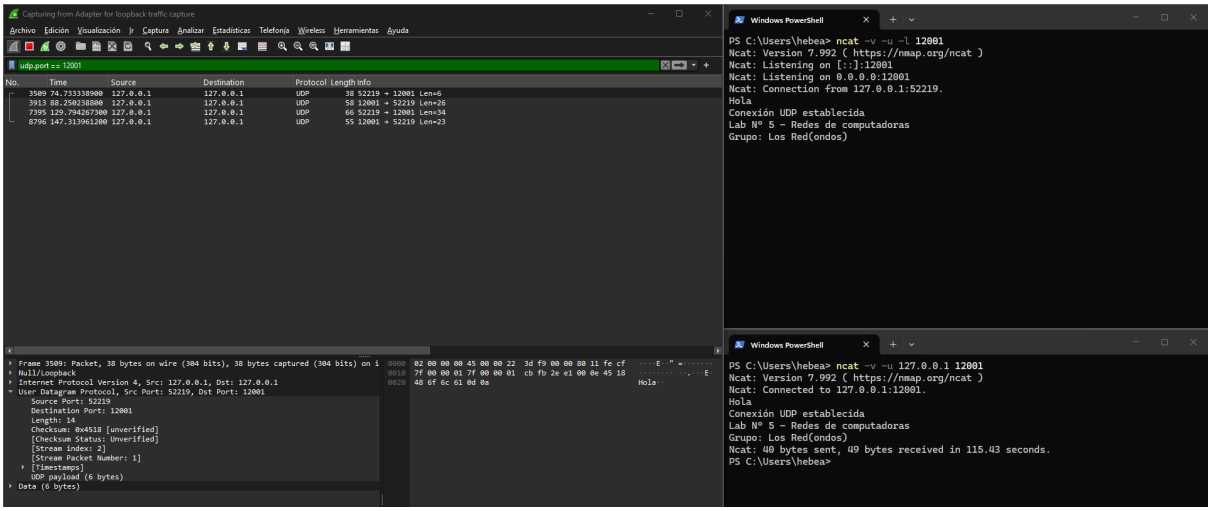
Experimento UDP.

Nueva captura en loopback, ahora con el filtro udp.port == 12001:

```
# Terminal A (servidor)
ncat -v -u -l 12001
```

```
# Terminal B (cliente)
ncat -v -u 127.0.0.1 12001
```

Envíen mensajes en ambos sentidos (el servidor puede responder una vez que recibió el primer datagrama del cliente) y cierren con Ctrl+C.



a)

¿Qué pasó en la red cuando ejecutaron el comando del cliente, antes de escribir el primer mensaje? Compáralo con TCP.

Cuando ejecutamos el comando del cliente, antes de escribir el primer mensaje, no se generó ningún mensaje o paquete, puesto que UPD es no orientado a conexión por lo que no verifica ningún intercambio de la misma. En cambio, en TCP, se pudo avistar el Three-way Handshake que verifica la conexión previa para enviar datos.

¿Cuántos datagramas generó cada mensaje? ¿Hay algo parecido a un ACK?

Cada mensaje generó 1 datagrama UDP de origen-destino, sin esperar ningún tipo de ACK puesto que UDP no se enfoca en recepción, sino únicamente en el envío de paquetes a la red sin esperar confirmación de recibido.

b)

Comparen el encabezado UDP con el encabezado TCP de un segmento con datos: ¿qué campos tiene cada uno? ¿Cuántos bytes ocupa cada encabezado?

El encabezado UDP tiene notoriamente menos campos que el TCP, principalmente debido a que toma las mismas medidas de aseguramiento de recepción de mensaje. UDP posee 8 bytes compuestos por:

- Source Port
- Destination Port
- Length
- Checksum

En cambio, en TCP el encabezado se compone de mínimo 20 bytes, los cuales abarcan:

- Source and Destination Ports.
- Sequence number.
- Acknowledgement Number.
- Header length.
- Flags.
- Windows size.
- Checksum.
- Options.

c)

¿Qué pasó en la red al cerrar el cliente con Ctrl+C? ¿Y en TCP?

Al querer cerrar el cliente en la red con CTRL+c en UPD, el proceso finaliza abruptamente, y la conexión se corta sin dar aviso al servidor, de manera local.

Por lo contrario en TCP se dió aviso del cierre de conexión que posteriormente llevó al Four-Way Wavehand enviando la flag de FIN, y recibiendo a su vez por el receptor su propio ACK y FIN.

d)

Para enviar la misma frase, ¿cuántos paquetes necesitaron en total con TCP y cuántos con UDP? ¿Qué "compran" con los paquetes extra de TCP?

Para enviar la misma frase, se necesitaron en UPD únicamente 2 paquetes, el de cliente y el de servidor. Mientras tanto, en TCP, se requirieron aproximadamente 9 paquetes para el establecimiento, cierre de conexión, y transferencia de datos en la red. Sin embargo, es importante tener en cuenta que esos paquetes demás que usó

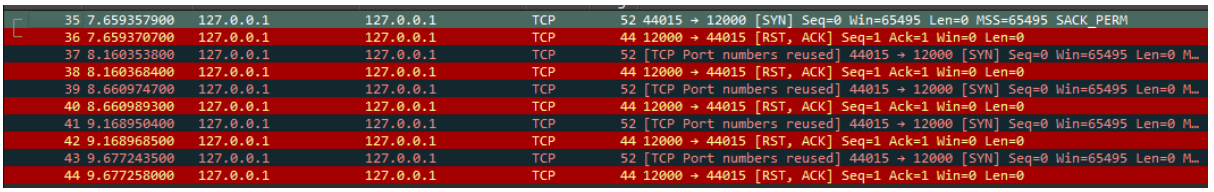
TCP aseguran una conexión más segura, controlada ante saturaciones o demoras, y que permite detectar y descartar duplicados.

e)

¿Y si nadie escucha? Con Wireshark capturando en loopback y el filtro tcp.port == 12000 || udp.port ==12001 || icmp, sin servidores corriendo:

```
ncat -v 127.0.0.1 12000          # TCP a un puerto cerrado
ncat -v -u 127.0.0.1 12001       # UDP a un puerto cerrado, escribir y enter
```

Al ejecutar la prueba con ncat -v 127.0.0.1 12000 podemos observar el mensaje en consola:
“Ncat: No se puede establecer una conexión ya que el equipo de destino denegó expresamente dicha conexión.”
Lo que significa que nuestro comando intento comunicarse con el puerto 12000 pero como ningún servidor lo estaba escuchando, se rechaza la conexión activa.
Además en wireshark se detectaron los siguientes paquetes:

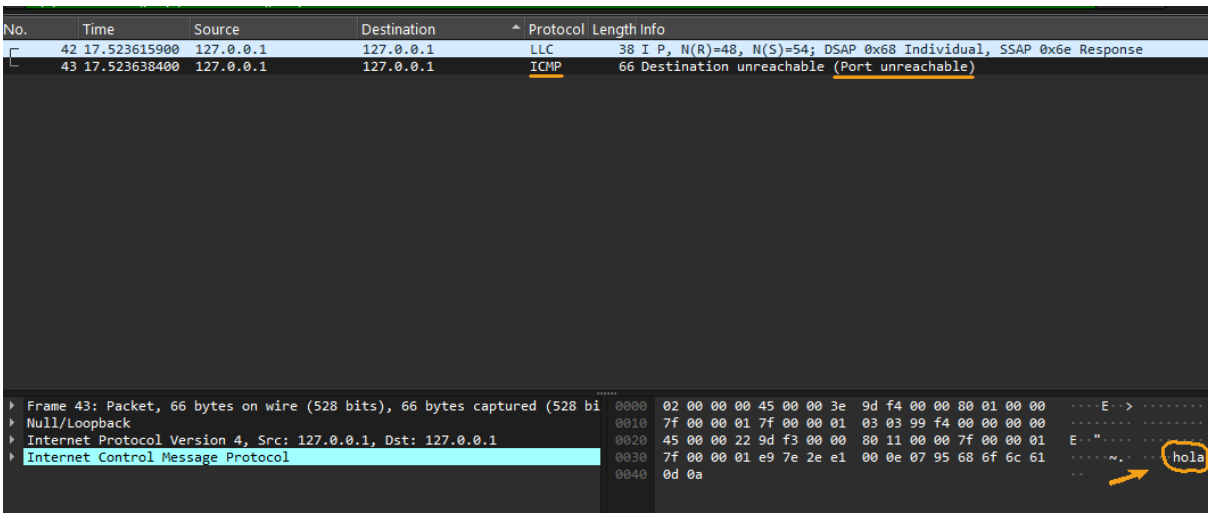


35	7.659357900	127.0.0.1	127.0.0.1	TCP	52 44015 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM
36	7.659370700	127.0.0.1	127.0.0.1	TCP	44 12000 → 44015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
37	8.160353800	127.0.0.1	127.0.0.1	TCP	52 [TCP Port numbers reused] 44015 → 12000 [SYN] Seq=0 Win=65495 Len=0 M…
38	8.160368400	127.0.0.1	127.0.0.1	TCP	44 12000 → 44015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
39	8.660974700	127.0.0.1	127.0.0.1	TCP	52 [TCP Port numbers reused] 44015 → 12000 [SYN] Seq=0 Win=65495 Len=0 M…
40	8.660989300	127.0.0.1	127.0.0.1	TCP	44 12000 → 44015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
41	9.168950400	127.0.0.1	127.0.0.1	TCP	52 [TCP Port numbers reused] 44015 → 12000 [SYN] Seq=0 Win=65495 Len=0 M…
42	9.168968500	127.0.0.1	127.0.0.1	TCP	44 12000 → 44015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
43	9.677243500	127.0.0.1	127.0.0.1	TCP	52 [TCP Port numbers reused] 44015 → 12000 [SYN] Seq=0 Win=65495 Len=0 M…
44	9.677258000	127.0.0.1	127.0.0.1	TCP	44 12000 → 44015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

El paquete gris con SYN flag es el intento inicial del cliente por iniciar la conexión hacia el puerto 12000.

Los paquetes rojos con RST y ACK flags son las respuestas indicando que el puerto está cerrado y la conexión denegada. Hay varios porque ncat realiza intentos automáticos de conexión al no recibir una respuesta exitosa al primer SYN.

Al ejecutar la prueba: “ncat -v -u 127.0.0.1 12001” la consola responde “ Ncat: Connected to 127.0.0.1:12001” Como UDP no esta orientado a conexión el comando simplemente asocia la dirección y el puerto de destino en el socket, entonces la consola nos dice:
“Ncat: Connected to 127.0.0.1:12001.” sin importar si hay alguien escuchando del otro lado o no.
Cuando escribimos un “hola” en la consola y lo enviamos podemos observar en wireshark dos paquetes.



el primer paquete con payload “hola” la consola envía un datagrama UDP al puerto 12001 y como UDP no verifica si hay alguien del otro lado escuchando antes de enviar el paquete sale igual y el payload lleva el “hola”.
En **el segundo paquete** ICMP Destination Unreachable el sistema recibe el datagrama UDP dirigido a un puerto donde ningún proceso está escuchando. Automáticamente genera un mensaje de error protocolo ICMP de tipo “Puerto inaccesible”.
Dentro del mismo paquete ICMP aparece aun así el payload “hola” también porque como respondimos en preguntas anteriores cuando se genera el mensaje ICMP de error incluye dentro de su estructura los bytes iniciales del paquete inicial que causó el problema para que el mismo emisor sepa exactamente qué paquete falló.

Además cuando la consola recibe el paquete ICMP de error fruto de enviar datos por UDP a un puerto cerrado nos tira: “Ncat: Se ha forzado la interrupción de una conexión existente por el host remoto.”.
Lo cual nos confirma que a pesar de que inicialmente nos dijo que la conexión fue establecida en realidad nunca se confirmó y al recibir este paquete de error la herramienta fuerza la interrupción de la comunicación.

Entonces en conclusión si nadie está escuchando, el protocolo TCP y UDP tienen comportamientos distintos.
TCP es un protocolo conversacional y estricto que te avisa de inmediato y de forma directa si la puerta está cerrada con un RST.
UDP en cambio es un protocolo ciego en ese sentido, manda el paquete sin preguntar y solo nos enteramos que fallo si el sistema de red o receptor nos avisa mediante un error ICMP.

4)

Servidor TCP mínimo - TP5 Redes de Computadoras.

Un socket es el "punto final" de una comunicación que el sistema operativo le entrega a un programa: el programa escribe y lee bytes en el socket, y el sistema operativo se ocupa de TCP/UDP, IP y la placa de red. No vamos a entrar en tanta teoría acá, ya está bastante pesado el TP. A programar.
Ahora hacemos nosotros lo que hacía ncat. Los scripts están resueltos para que puedan concentrarse en relacionar cada línea con lo que ocurre en la red.

a)

Leer el código del servidor:

```
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 12000

# 1. Crear el socket. AF_INET = IPv4, SOCK_STREAM = TCP.
#    Esto es una operación local: todavía no viaja nada por la red.
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Permite reiniciar el servidor enseguida sin el error "Address already in use"
# (ver TIME_WAIT en la guía). No es imprescindible para entender el resto.
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# 2. Asociar el socket a una IP y un puerto locales (local, sin tráfico).
servidor.bind((HOST, PORT))

# 3. Pedirle al sistema operativo que acepte conexiones en ese puerto.
#    A partir de acá, un SYN que llegue a este puerto será respondido con SYN-ACK.
servidor.listen(1)
print(f"[servidor TCP] escuchando en {HOST}:{PORT} ...")

# 4. Esperar a que un cliente complete el three-way handshake.
#    accept() devuelve un socket NUEVO, dedicado a esta conexión,
#    y la dirección (IP, puerto) del cliente.
conexion, direccion_cliente = servidor.accept()
print(f"[servidor           TCP]           conexión           aceptada           desde
{direccion_cliente[0]}:{direccion_cliente[1]}")

# 5. Recibir hasta 1024 bytes. TCP entrega un flujo de bytes: recv() puede
#    devolver menos (o más mensajes juntos) de lo que el cliente envió en un send().
datos = conexion.recv(1024)

# 6. Mostrar lo recibido (bytes -> texto).
mensaje = datos.decode("utf-8", errors="replace")
print(f"[servidor TCP] recibido ({len(datos)} bytes): {mensaje!r}")

# 7. Enviar una respuesta sencilla.
respuesta = f"Recibido: {mensaje}"
conexion.sendall(respuesta.encode("utf-8"))
print(f"[servidor TCP] respuesta enviada: {respuesta!r}")

# Esperar a que el cliente cierre: recv() devuelve b"" cuando llega su FIN.
while conexion.recv(1024):
```

```
    pass
print("[servidor TCP] el cliente cerró la conexión")

conexion.close()
servidor.close()
```

Uso:

```
python tcp_server.py            # escucha en 127.0.0.1:12000 (solo esta computadora)
python tcp_server.py 0.0.0.0         # escucha en todas las interfaces (Parte 7)
python tcp_server.py 0.0.0.0 12000   # IP y puerto explícitos
```

b)

Leer el código del cliente:

```
python tcp_client.py                          # se conecta a 127.0.0.1:12000
python tcp_client.py 192.168.43.20            # IP del servidor (Parte 7)
python tcp_client.py 192.168.43.20 "Hola!"    # IP y mensaje
```

Uso:

```
import socket
import sys

SERVER_IP = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
MENSAJE = sys.argv[2] if len(sys.argv) > 2 else "Hola servidor"
SERVER_PORT = 12000

# 1. Crear el socket TCP (operación local).
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.settimeout(5)  # evita quedar bloqueado para siempre si el servidor no responde

# 2. Conectarse: acá el sistema operativo envía el SYN y completa el
#    three-way handshake. El puerto local del cliente lo elige el SO.
try:
    cliente.connect((SERVER_IP, SERVER_PORT))
except ConnectionRefusedError:
    sys.exit("[cliente TCP] conexión rechazada: el host respondió, pero nadie escucha en
ese puerto (RST)")
except (socket.timeout, TimeoutError):
```

```
        sys.exit("[cliente TCP] sin respuesta al intento de conexión (¿firewall? ¿IP
equivocada? ¿aislamiento de clientes?)")

ip_local, puerto_local = cliente.getsockname()
print(f"[cliente      TCP]      conectado      a      {SERVER_IP}:{SERVER_PORT}      desde
{ip_local}:{puerto_local}")

# 3. Enviar el mensaje (texto -> bytes).
cliente.sendall(MENSAJE.encode("utf-8"))
print(f"[cliente TCP] enviado: {MENSAJE!r}")

# 4. Recibir la respuesta.
datos = cliente.recv(1024)

# 5. Mostrarla.
print(f"[cliente    TCP]    respuesta    ({len(datos)}    bytes):    {datos.decode('utf-8',
errors='replace')!r}")

# Cerrar: el SO envía un FIN y comienza el cierre de la conexión.
cliente.close()
```

c)

Ejecutar (capturando en loopback con el filtro tcp.port == 12000):

```
# Terminal A
python tcp_server.py

# Terminal B
python tcp_client.py 127.0.0.1 "Hola desde mi programa"
```

Comportamiento esperado (el puerto del cliente va a ser otro):

[servidor TCP] escuchando en 127.0.0.1:12000 ...
[servidor TCP] conexión aceptada desde 127.0.0.1:48096
[servidor TCP] recibido (22 bytes): 'Hola desde mi programa'
[servidor TCP] respuesta enviada: 'Recibido: Hola desde mi programa'
[servidor TCP] el cliente cerró la conexión

[cliente TCP] conectado a 127.0.0.1:12000 desde 127.0.0.1:48096
[cliente TCP] enviado: 'Hola desde mi programa'
[cliente TCP] respuesta (32 bytes): 'Recibido: Hola desde mi programa'

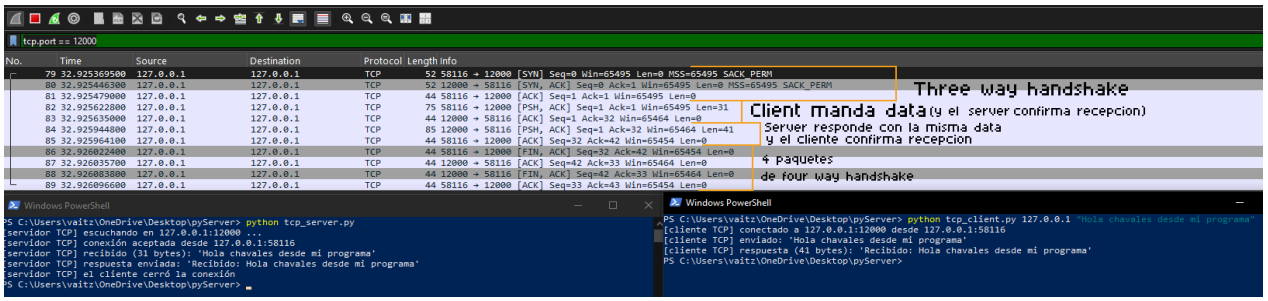
El servidor atiende un cliente y termina. Para repetir, vuelvan a ejecutarlo.

d)

Con la captura, completar una tabla como la siguiente. Para cada llamada, indicar si genera tráfico y, si lo genera, qué segmentos (número de paquete en Wireshark y flags):

Llamada	¿Dónde se ejecuta?	¿Genera tráfico?	Segmentos que observan
socket()	servidor y cliente	no	0
bind()	servidor	no	0
listen()	servidor	no	0
connect()	cliente	si	se observan 3 paquetes, representando el three way handshake.
accept()	servidor	no	no genera paquetes ni tráfico realmente, el kernel del servidor es el que realmente completa el handshake de red de forma automática. La función accept() lo único que hace es tomar esa conexión ya establecida y entregarle al programa un socket nuevo y dedicado exclusivamente a ese cliente.
sendall()	servidor y servidor	si	2 paquetes, uno con el payload y otro de confirmación de recepción.
recv()	servidor	no	esta función sólo lee memoria, la lógica de recibir un dato y mandar confirmaciones lo hace

			el protocolo tcp. Asi que la función realmente no manda ningún paquete ni genera trafico.
close()	ambos	si	4 paquetes. el four way handshake.



se pueden observar tráfico que se puede dividir en apertura/clausura de la conexión y envío de información y paquetes de confirmación tal y como se muestra en la imagen.